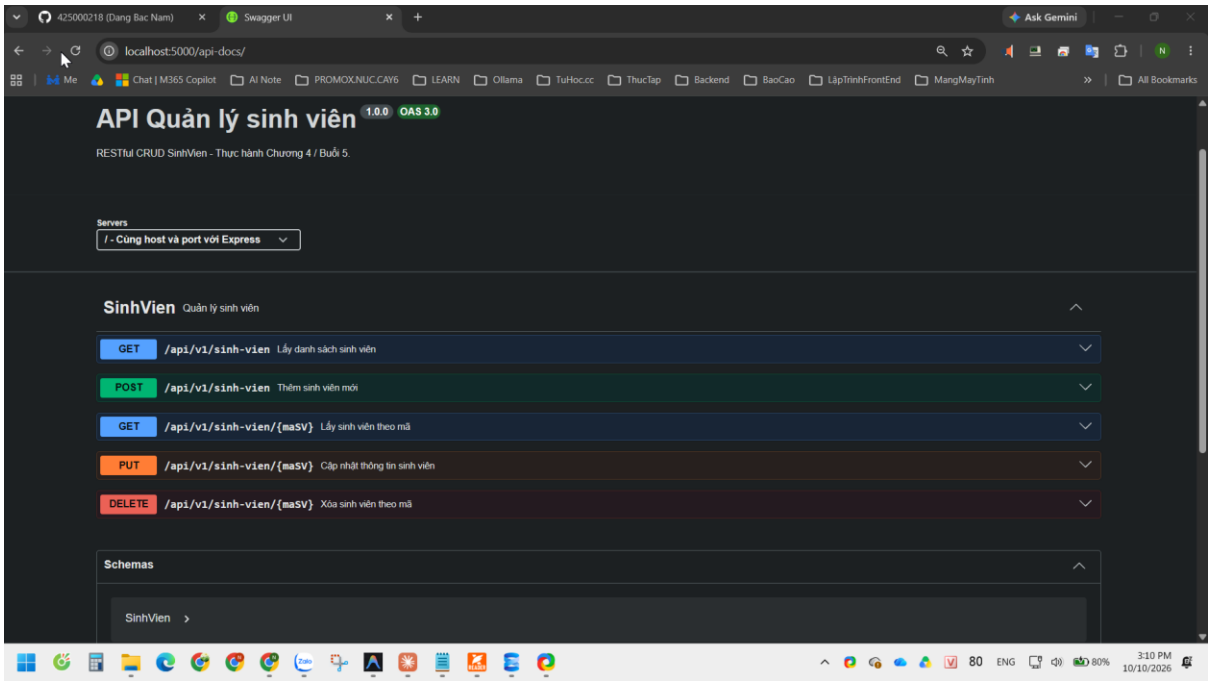
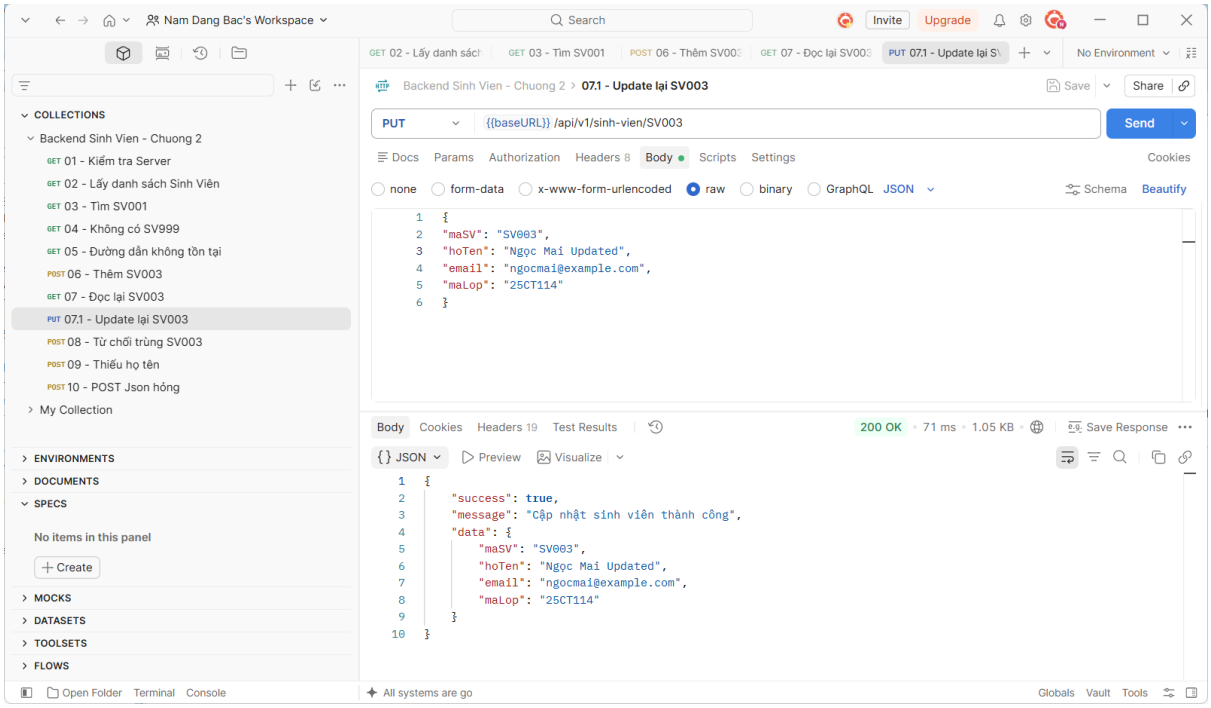
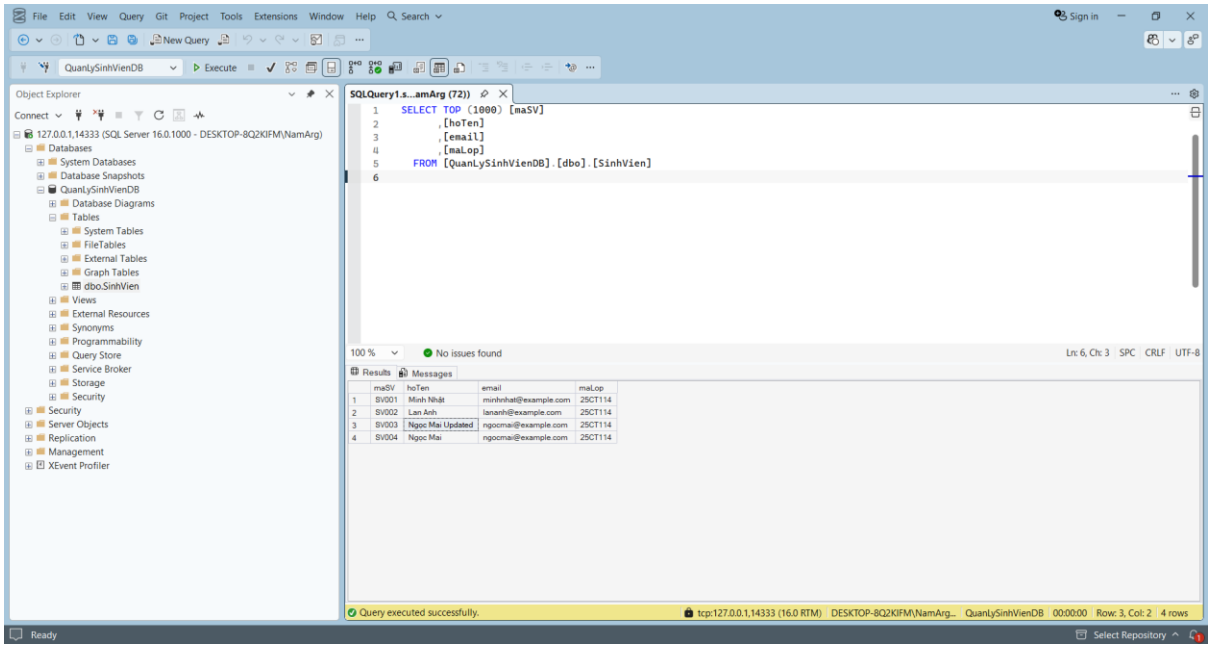
CHẠY TEST TRÊN POSTMAN CHƯƠNG 4

swagger-operations.png

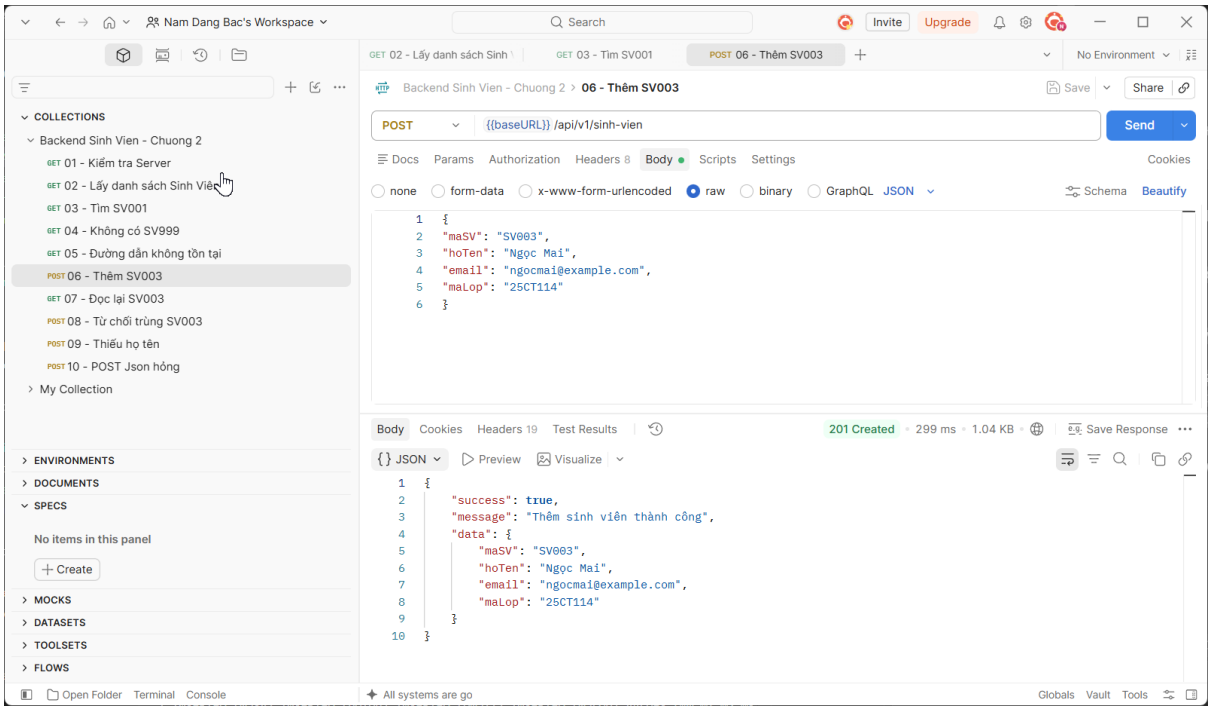


post-created.png

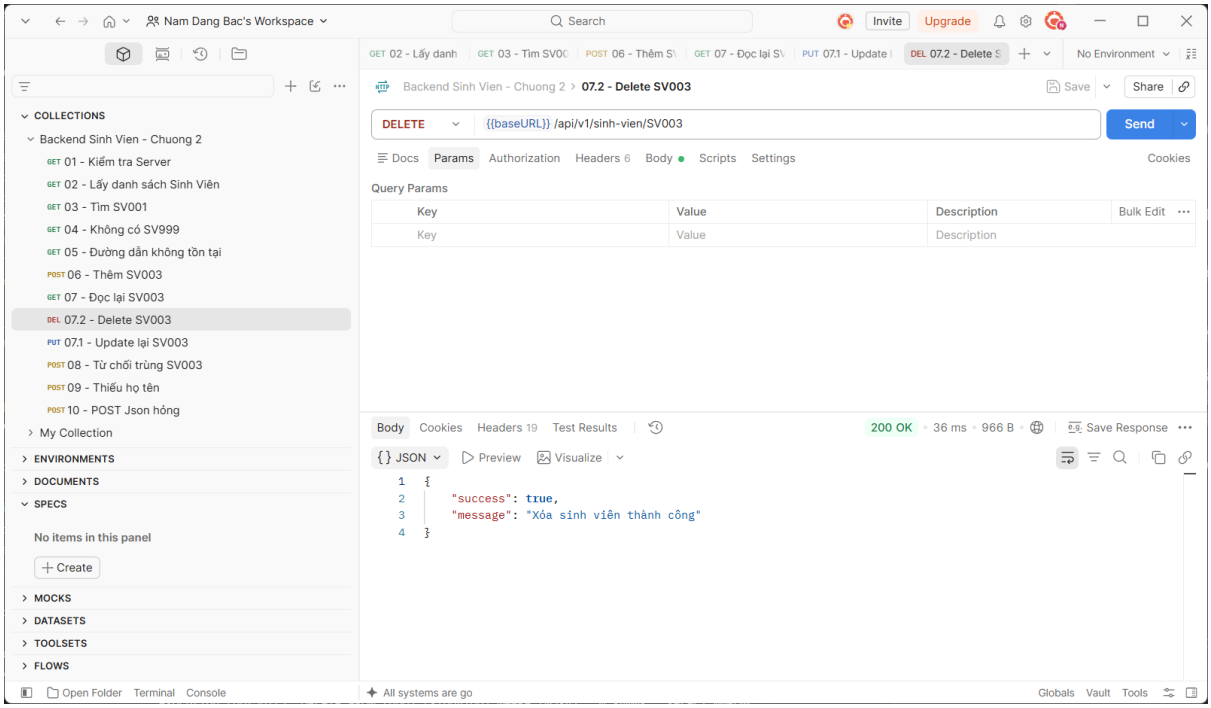


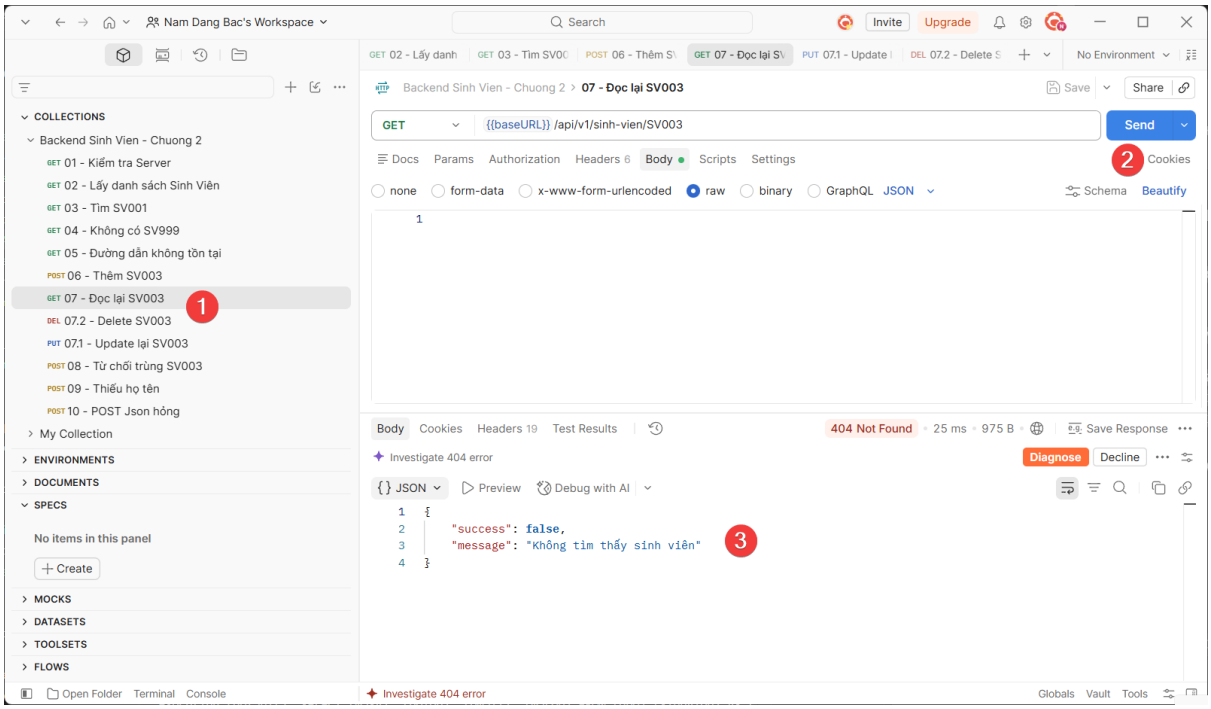


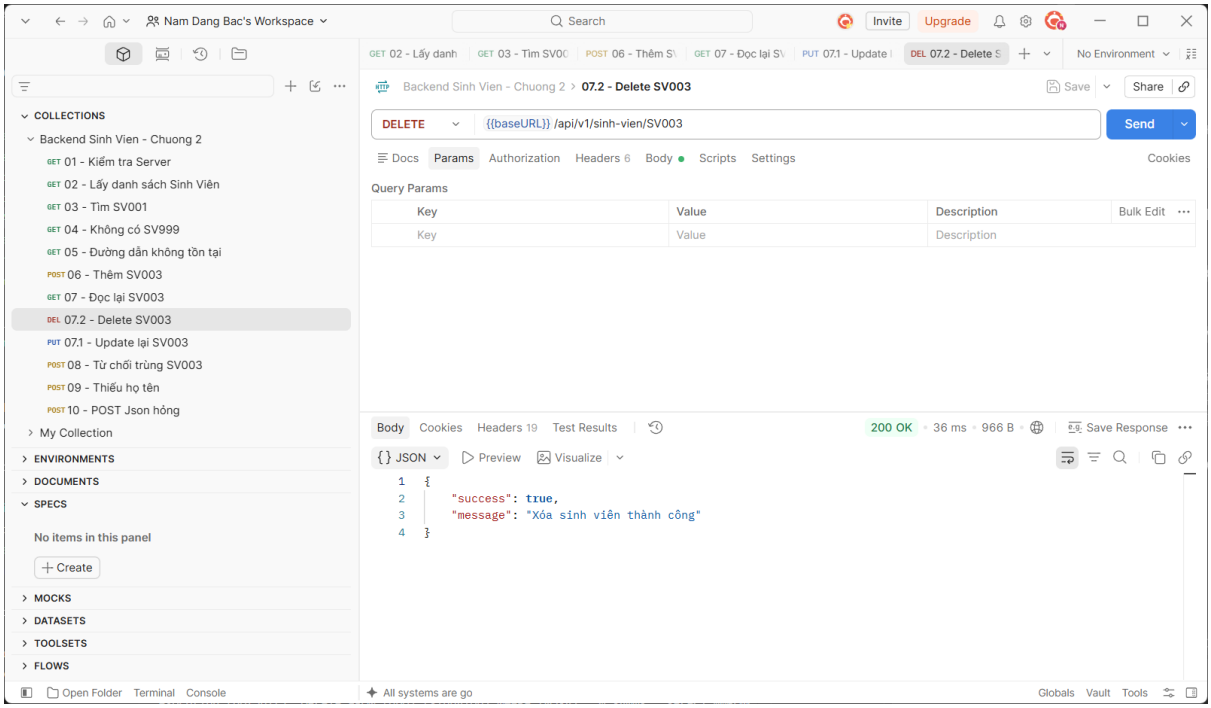
put-verified.png



deleted-get-404.png







NHẬT KÝ PROMPT & KỸ NĂNG PROMPT ENGINEERING

BÀI THỰC HÀNH BUỔI 5 / CHƯƠNG 4: RESTful CRUD & SWAGGER UI

- **Học phần:** Lập trình Backend NodeJS & TypeScript
- **Dự án:** Quản lý sinh viên (RESTful CRUD)
- **Phương pháp:** Vibe Coding theo khung CTCO (Context - Task - Constraint - Output)
- **Môi trường:** Node.js v24, Express 5, TypeScript ESM, Sequelize 6, SQL Server

1. Khung Prompt Engineering CTCO là gì?

Khung CTCO là phương pháp chuẩn hóa cấu trúc prompt giúp lập trình viên giao tiếp chính xác, rõ ràng và hiệu quả với AI Assistant trong quá trình phát triển phần mềm:

Thành phần CTCO	Ý nghĩa & Câu hỏi tự đặt ra trước khi gửi AI
C - Context **(Bối cảnh)**	Tôi đang làm dự án gì? Công nghệ và file hiện có là gì? (Ví dụ: Express 5, TypeScript ESM NodeNext, Sequelize 6, SQL Server, các file routes, controller, service hiện tại).
T - Task **(Nhiệm vụ)**	Tôi muốn AI làm một việc cụ thể nào? (Chỉ giải quyết một tác vụ đơn mục tiêu trong mỗi prompt, ví dụ: viết riêng hàm PUT hoặc tạo file openapi.yaml).
C - Constraint **(Ràng buộc)**	Điều gì bắt buộc? Điều gì AI không được tự ý thay đổi? (Giữ nguyên phân tầng Route - Controller - Service, giữ mã lỗi HTTP 400/404/409, không tự ý chuyển CommonJS hay đổi kiểu dữ liệu).
O - Output **(Đầu ra)**	Tôi muốn nhận kết quả như thế nào? (Chỉ ra đường dẫn file cần sửa, code chi tiết có comment tiếng Việt và hướng dẫn các bước kiểm thử cụ thể).

2. Nhật ký Prompt thực tế các bước phát triển

Prompt 1: Bổ sung chức năng cập nhật (PUT /api/v1/sinh-vien/:maSV)

• **Context:** Dự án Quản lý sinh viên Express 5 + TypeScript ESM NodeNext, kết nối SQL Server qua Sequelize 6. Đã có 3 API: GET danh sách, GET chi tiết, POST thêm mới.

• **Task:** Bổ sung chức năng PUT /api/v1/sinh-vien/:maSV để cập nhật thông tin sinh viên hiện có.

• **Constraint:** Lấy maSV từ URL params. Body chỉ nhận 3 trường hoTen, email, maLop (không cho đổi maSV qua body). Validation chuỗi không rỗng, regex email, độ dài SQL (hoTen <= 150, email <= 254, maLop <= 50) sai trả 400; không tìm thấy trả 404; thành công trả 200. Giữ nguyên kiến trúc 3 tầng.

• **Output:** Code hoàn chỉnh cho sinhVien.service.ts, sinhVien.controller.ts, sinhVien.routes.ts và hướng dẫn test bằng Postman.

• **Đánh giá & Kết quả:** AI sinh đúng luồng async/await, Controller có await, kiểm thử T06 (200) và T07 (400 khi thiếu email) hoạt động chính xác.

Prompt 2: Bổ sung chức năng xóa (DELETE /api/v1/sinh-vien/:maSV)

• **Context:** Tiếp tục dự án trên với 4 API đã chạy ổn định.

• **Task:** Bổ sung chức năng DELETE /api/v1/sinh-vien/:maSV để xóa sinh viên khỏi CSDL.

• **Constraint:** Lấy maSV từ URL, không nhận body. Tìm qua SinhVien.findByPk(maSV), nếu không thấy trả 404; xóa thành công trả 200 với { "success": true, "message": "Xóa sinh viên thành công" } (không có trường data). Không ảnh hưởng các API khác.

• **Output:** Code cho 3 tầng và các bước gọi GET lại để kiểm chứng dữ liệu đã biến mất thật sự.

• **Đánh giá & Kết quả:** Xóa thành công SV003, gọi lại GET trả về đúng 404 Not Found chứng minh bản ghi đã xóa khỏi SQL Server.

Prompt 3: Tạo file đặc tả OpenAPI YAML riêng biệt (docs/openapi.yaml)

• **Context:** Hệ thống đã hoàn thiện đủ 5 API CRUD sinh viên.

• **Task:** Tạo file docs/openapi.yaml chuẩn OpenAPI 3.0.3 mô tả toàn bộ 5 API.

• **Constraint:** Tách riêng file YAML, không dùng JSDoc hay swagger-jsdoc quét comment trong code. Tái sử dụng components/$ref cho schemas và responses. Mô tả chính xác các status code 200, 201, 400, 404, 409, 500.

• **Output:** Nội dung file docs/openapi.yaml hoàn chỉnh, thụt lề chuẩn cú pháp YAML (dùng space, không tab).

Prompt 4: Tích hợp Swagger UI vào Express Server

• **Context:** Đã có file docs/openapi.yaml trong dự án Express + TypeScript.

• **Task:** Tích hợp Swagger UI tại endpoint /api-docs.

• **Constraint:** Tạo src/config/swagger.config.ts đọc file YAML bằng js-yaml an toàn. Mount Swagger UI trong server.ts trước middleware 404. Giữ nguyên cấu trúc dự án.

• **Output:** Code hoàn chỉnh cho swagger.config.ts và đoạn cấu hình trong server.ts.

3. Bài tự viết Prompt mở rộng (Không sao chép)

Yêu cầu: Vận dụng khung CTCO để yêu cầu AI bổ sung tính năng tìm kiếm danh sách sinh viên theo mã lớp (GET /api/v1/sinh-vien/lop/:maLop).

NỘI DUNG PROMPT CTCO: **CONTEXT:** Dự án Quản lý sinh viên Express 5 + TypeScript ESM, kết nối SQL Server bảng dbo.SinhVien qua Sequelize Model SinhVien. Hệ thống đã có đủ 5 API CRUD cơ bản. **TASK:** Bổ sung thêm một API tìm kiếm danh sách sinh viên theo mã lớp: GET /api/v1/sinh-vien/lop/:maLop. **CONSTRAINT:** 1. Endpoint: GET /api/v1/sinh-vien/lop/:maLop, maLop lấy từ req.params. 2. Service: Sử dụng SinhVien.findAll({ where: { maLop }, order: [['maSV', 'ASC']] }). 3. Phản hồi: Luôn trả HTTP 200 với data là mảng các sinh viên thuộc lớp đó (nếu lớp chưa có sinh viên nào thì trả mảng rỗng []). 4. Không làm xung đột với route GET /:maSV hiện tại (phải khai báo route /lop/:maLop trước /:maSV trong sinhVien.routes.ts để tránh Express hiểu nhầm chuỗi 'lop' là một maSV). 5. Giữ nguyên toàn bộ 5 API CRUD cũ và cấu trúc DTO hiện có. **OUTPUT:** Cung cấp code cần sửa trong sinhVien.service.ts, sinhVien.controller.ts, sinhVien.routes.ts và một test case Postman mẫu để kiểm thử.

• **Giải thích vì sao chọn endpoint /api/v1/sinh-vien/lop/:maLop:** Việc bổ sung tiền tố "lop/" giúp phân định rõ ràng giữa thao tác tra cứu một tài nguyên cá nhân cụ thể theo khóa chính (/:maSV) và thao tác lọc danh sách theo thuộc tính nhóm (/lop/:maLop).

Đồng thời, khai báo route này đứng trước /:maSV trong file Routes giúp Express Router không bị nhận diện nhầm chuỗi "lop" là một giá trị của tham số :maSV.

4. Giải đáp 6 câu hỏi vấn đáp cốt lõi cuối buổi

Câu 1: Request PUT đi qua những file nào trong dự án?

Request PUT /api/v1/sinh-vien/:maSV đi qua tuần tự theo mô hình phân tầng:

1. **src/server.ts**: Tiếp nhận HTTP request, đi qua các middleware toàn cục (logger, helmet, cors, express.json) rồi định tuyến vào /api/v1/sinh-vien.

2. **src/routes/sinhVien.routes.ts**: Khớp phương thức PUT và pattern /:maSV, chuyển giao việc xử lý cho controller capNhatSinhVien.

3. **src/controllers/sinhVien.controller.ts**: Đọc maSV từ req.params, lấy body dữ liệu và gọi await capNhatSinhVienService(maSV, req.body).

4. **src/services/sinhVien.service.ts**: Thực hiện kiểm tra tính hợp lệ của dữ liệu (validation), tìm bản ghi qua SinhVien.findByPk và gọi sv.update({ hoTen, email, maLop }).

5. **src/models/SinhVien.ts & SQL Server**: Sequelize ORM sinh câu lệnh UPDATE dbo.SinhVien SET ... WHERE maSV = ... thực thi xuống SQL Server.

6. Sau khi cập nhật thành công, Service trả về DTO -> Controller đóng gói JSON { success: true, message: ..., data } và gửi trả HTTP 200 cho Client.

Câu 2: Vì sao gọi GET sau khi DELETE trả về 404 là một test đạt?

Bởi vì mục tiêu của thao tác DELETE là xóa bỏ vĩnh viễn tài nguyên khỏi hệ thống.

Việc gọi GET ngay sau DELETE và nhận được HTTP 404 Not Found chính là bằng chứng xác minh độc lập (verification) chứng minh rằng bản ghi đã thực sự bị xóa khỏi CSDL SQL Server chứ không phải server chỉ trả về mã 200 giả vờ thành công.

Nếu GET lại vẫn trả về 200 thì bài test xóa xem như thất bại.

Câu 3: File Swagger YAML (docs/openapi.yaml) có tự xử lý nghiệp vụ Backend không?

Không. File YAML chỉ đóng vai trò là tài liệu đặc tả hợp đồng giao tiếp (API Contract Specification).

Nó mô tả cho Client biết API có những endpoint nào, nhận tham số gì và cấu trúc dữ liệu trả về ra sao.

Toàn bộ logic kiểm tra hợp lệ, xử lý nghiệp vụ, kiểm tra ràng buộc CSDL và lưu trữ thực tế hoàn toàn do mã nguồn Backend (Route, Controller, Service, Sequelize) đảm nhiệm lúc chạy.

Câu 4: Vì sao phải gọi GET lại sau mỗi thao tác ghi (POST, PUT, DELETE)?

Một API có thể phản hồi mã HTTP 200 hoặc 201 thành công cùng dữ liệu mẫu dạng JSON nhưng thực tế câu lệnh lưu vào CSDL có thể đã bị bỏ qua hoặc bị lỗi ngầm.

Việc chủ động gọi GET lại sau mỗi thao tác ghi giúp đối chiếu trạng thái thực tế của dữ liệu trong CSDL, đảm bảo tính nhất quán (Data Consistency) và chứng minh thao tác ghi có hiệu lực thật sự.

Câu 5: Sau khi restart server Node.js, dữ liệu có còn không?

Dữ liệu vẫn còn nguyên vẹn 100%.

Lý do là dự án đã chuyển sang lưu trữ bền vững (Persistent Storage) trong hệ quản trị cơ sở dữ liệu Microsoft SQL Server trên ổ cứng. Khi khởi động lại server Node.js, chỉ có tiến trình ứng dụng trong bộ nhớ RAM được khởi động lại, còn toàn bộ dữ liệu sinh viên trong bảng dbo.SinhVien của SQL Server không hề bị ảnh hưởng.

Câu 6: Khung CTCO giúp viết Prompt rõ ràng và hiệu quả như thế nào?

Khung CTCO mang lại 4 lợi ích vượt trội:

• Cung cấp đầy đủ Context giúp AI hiểu rõ kiến trúc dự án, tránh việc AI tự suy diễn hoặc tạo mã nguồn không khớp phiên bản.

• Định nghĩa Task cụ thể giúp chia nhỏ bài toán lớn thành các bước giải quyết đơn mục tiêu, dễ kiểm soát và dễ debug.

• Thiết lập Constraint nghiêm ngặt để bảo vệ các nguyên tắc thiết kế quan trọng (không đổi kiểu dữ liệu, giữ cấu trúc DTO, không phá vỡ logic cũ).

• Quy định rõ Output giúp kết quả trả về sử dụng được ngay, có đầy đủ vị trí file cần chỉnh sửa và kịch bản kiểm thử rõ ràng.